CUDA Graph for Head Pipelining

All-head replay cuts launch gaps; head bands expose overlap

Request-stable shape

- One video keeps the same H/W/T and reference layout through denoising

All heads + Graph

- Cuts launch overhead, but attention still waits for the full exchange

Head bands + Graph

- Starts attention as each fixed band arrives and pipelines later bands

Capture boundary

- Graph: unpack → attention → pack; RDMA and readiness events stay outside



Same Graph Benefit · Different Overlap

Head batching exposes overlap; CUDA Graph keeps the finer pipeline from becoming launch-bound

Sources: CUDA Programming Guide · current H3 Ulysses prototype execution contract · author diagram

CUDA Graph Reuse Depends on the Video Profile

Arbitrary video requests have lower Graph reuse than LLM decode; fixed live streams reuse across continuation windows

Question	LLM Decode	H3 Video	Serving Policy
Shape key	Batch bucket q_len = 1	H/W/T · reference slots head band	Cache a small set of hot profiles
Replay window	Many token steps and requests	Across blocks + steps PDD8: 400 · Base50: 2,500*	Reuse persistent slots across continuation windows
VSA Top-K	—	Count fixed per profile indices and values change	Content changes replay the same Graph
New geometry	Select another batch bucket	Bucket hit or eager path	Capture only after the profile becomes hot
Buffer addresses	Static or pooled decode buffers	Registered QKV/O staging buffers	Use a persistent buffer pool

*Per band/profile GraphExec when configuration matches · (1,2,3,1) uses four GraphExecs · Sources: CUDA Programming Guide; current H3 prototype audit